

SUPMEAL

Documentation technique

Application web de gestion de recettes et de planification de repas

Maham BA

1. Contexte et objectifs du projet

SUPMEAL est une application web conçue comme une alternative complète et intuitive aux solutions existantes de gestion de recettes (Mealie, Tandoor Recipes, Paprika). Elle permet à chaque utilisateur de créer, importer, organiser et planifier des recettes de cuisine, seul ou au sein de « cookbooks » partagés avec des permissions différenciées.

Les fonctionnalités principales couvertes sont :

- Authentification (email/mot de passe et OAuth2 Google)
- Gestion de recettes personnelles et partagées (CRUD complet, photo, ingrédients, tags)
- Cookbooks collaboratifs avec système d'invitation et rôles (propriétaire, éditeur, lecteur, commentateur)
- Recherche et filtrage multi-critères (mots-clés, tags, favoris)
- Planification de repas (MealPlan) personnelle ou partagée
- Messagerie instantanée de groupe et commentaires sur les recettes
- Import / export des données au format JSON
- Déploiement intégral via Docker Compose

2. Choix technologiques et justifications

2.1 Backend

Technologie	Rôle	Justification
Node.js + TypeScript	Runtime / langage	Typage statique réduisant les erreurs, écosystème riche, cohérence avec le frontend (même langage)
Express	Framework HTTP	Léger, très documenté, contrôle fin du routage et des middlewares
Prisma (ORM)	Accès base de données	Typage généré automatiquement depuis le schéma, migrations versionnées, requêtes sécurisées contre l'injection SQL
PostgreSQL	Base de données	SGBD relationnel robuste, gestion native des tableaux (allergies[]) et des contraintes d'intégrité
JWT (jsonwebtoken)	Authentification	Standard sans état (stateless), adapté à une architecture découplée frontend/backend
bcryptjs	Hachage des mots de passe	Algorithme éprouvé avec salage automatique, jamais de mot de passe stocké en clair
Passport.js	OAuth2	Bibliothèque standard pour l'authentification tierce (Google)
Socket.io	Temps réel	Messagerie instantanée par salon (room) lié à chaque cookbook, avec repli automatique si WebSocket indisponible
Zod	Validation	Validation stricte et typée des données entrantes, messages d'erreur structurés
Multer	Upload de fichiers	Gestion multipart/form-data pour les photos de recettes

2.2 Frontend

Technologie	Rôle	Justification
React + TypeScript	Bibliothèque UI	Composants réutilisables, typage partagé avec le backend, très large communauté
Vite	Outil de build	Démarrage et rechargement à chaud quasi instantanés, build de production optimisé
React Router	Routage	Navigation côté client standard pour une SPA
Zustand	Gestion d'état	Alternative légère à Redux pour l'état d'authentification global
Tailwind CSS v4	Styles	Design system par variables CSS (@theme), classes utilitaires rapides à itérer
Axios	Client HTTP	Intercepteurs pour l'injection automatique du token JWT sur chaque requête
Socket.io-client	Temps réel	Connexion persistante au serveur pour le chat et les mises à jour live
lucide-react	Icônes	Bibliothèque d'icônes cohérente et légère

2.3 Déploiement

Docker et Docker Compose orchestrent trois conteneurs indépendants (base de données, API, client web), conformément au cahier des charges. Chaque service dispose de son propre Dockerfile et communique via le réseau interne Docker.

3. Architecture Générale

L'application suit une architecture trois-tiers stricte : le client web ne contient aucune logique métier et ne fait que consommer l'API REST du serveur, qui seul dialogue avec la base de données via Prisma.

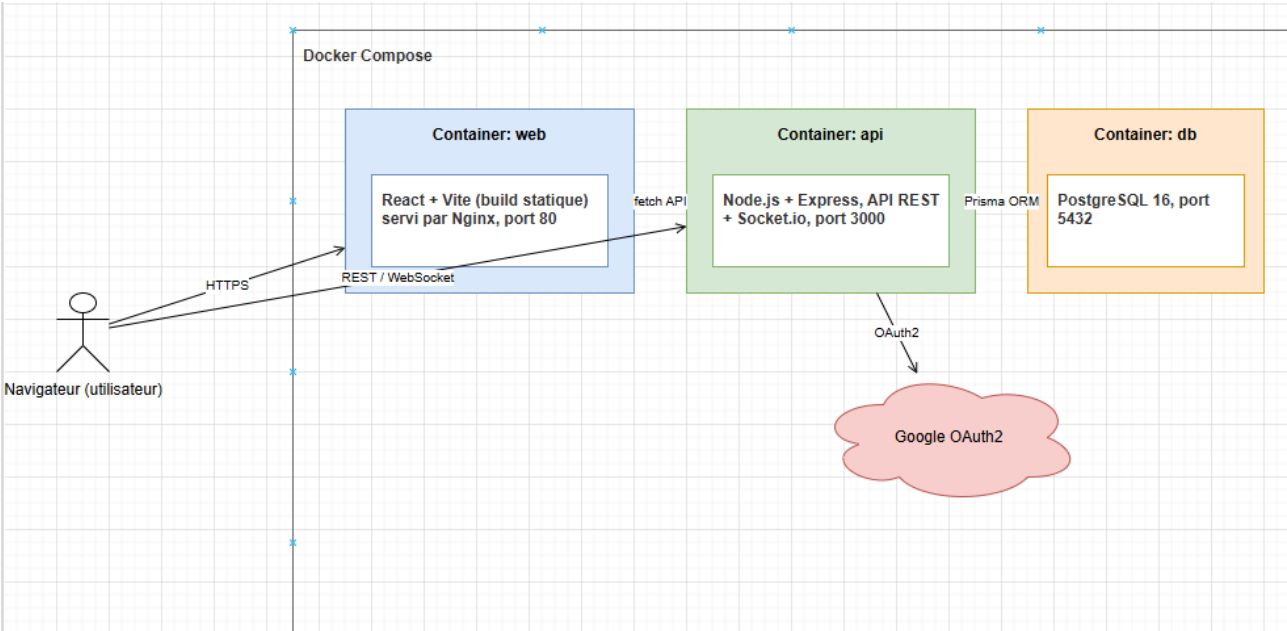


Figure 1 — Architecture de déploiement (conteneurs Docker)

Le client web (React) est compilé en fichiers statiques et servi par Nginx. Il communique avec l'API via des requêtes HTTP (REST) pour les opérations classiques, et via une connexion WebSocket (Socket.io) pour la

messaging instantané. L'API Node.js expose également une route dédiée à l'authentification OAuth2 Google.

4- Modèle de données

Le schéma relationnel comporte onze modèles principaux, gérés par Prisma. Les relations clefs sont les suivantes :

- Un User peut être auteur de plusieurs Recipe, membre de plusieurs Cookbook (via CookbookMember, avec un rôle et un statut d'invitation), et posséder plusieurs MealPlan.
- Une Recipe appartient à un seul emplacement à la fois : soit personnelle (cookbookId nul), soit rattachée à un Cookbook précis.
- CookbookMember porte le rôle (OWNER, EDITOR, READER, COMMENTER) et le statut (PENDING, ACCEPTED), implémentant le système d'invitation avec validation.
- MealPlanRecipe est une table de jointure enrichie reliant un MealPlan à une Recipe, avec une date, un type de repas et un nombre de portions propre à cette occurrence.

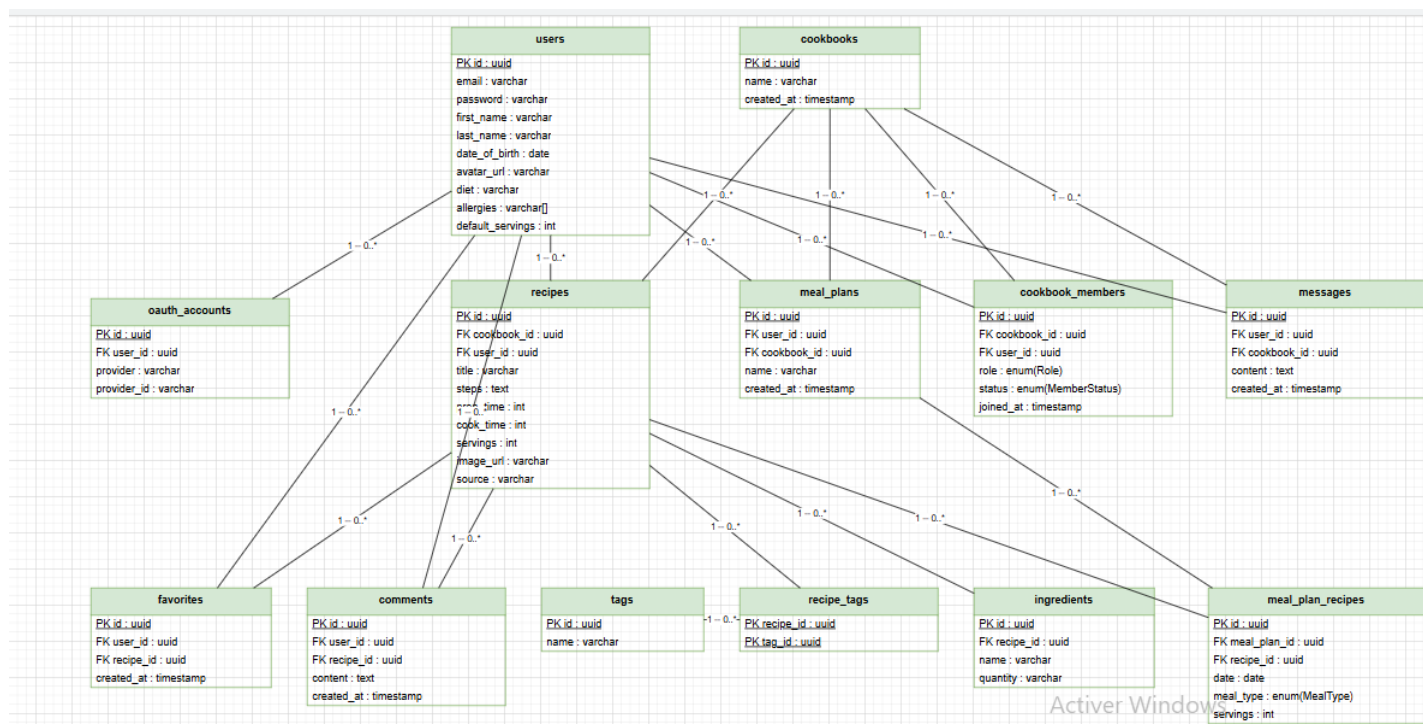


Figure 2 — Schéma entité-association de la base de données

5. Diagrammes UML

5.1 Diagramme de cas d'utilisation

Trois niveaux d'acteurs interagissent avec le système : un visiteur non authentifié, un utilisateur connecté, et un propriétaire de cookbook disposant de droits étendus de gestion des membres.

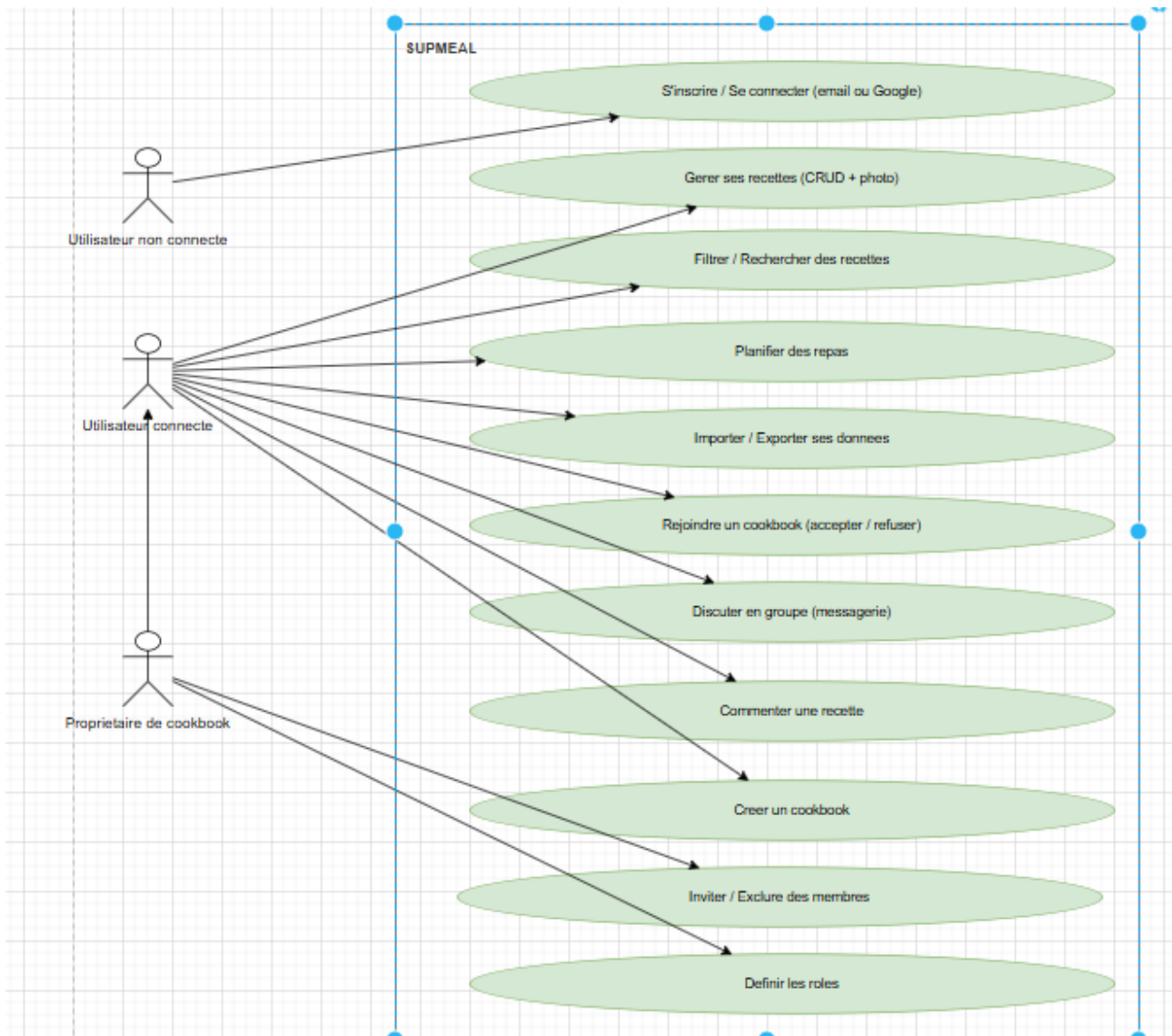


Figure 3 — Diagramme de cas d'utilisation

5.2 Diagramme de séquence - invitation à un cookbook

Ce flux illustre le mécanisme d'invitation avec validation explicite : l'ajout d'un membre crée une ligne au statut PENDING, invisible dans les listes actives tant que l'utilisateur invité n'a pas expressément accepté.

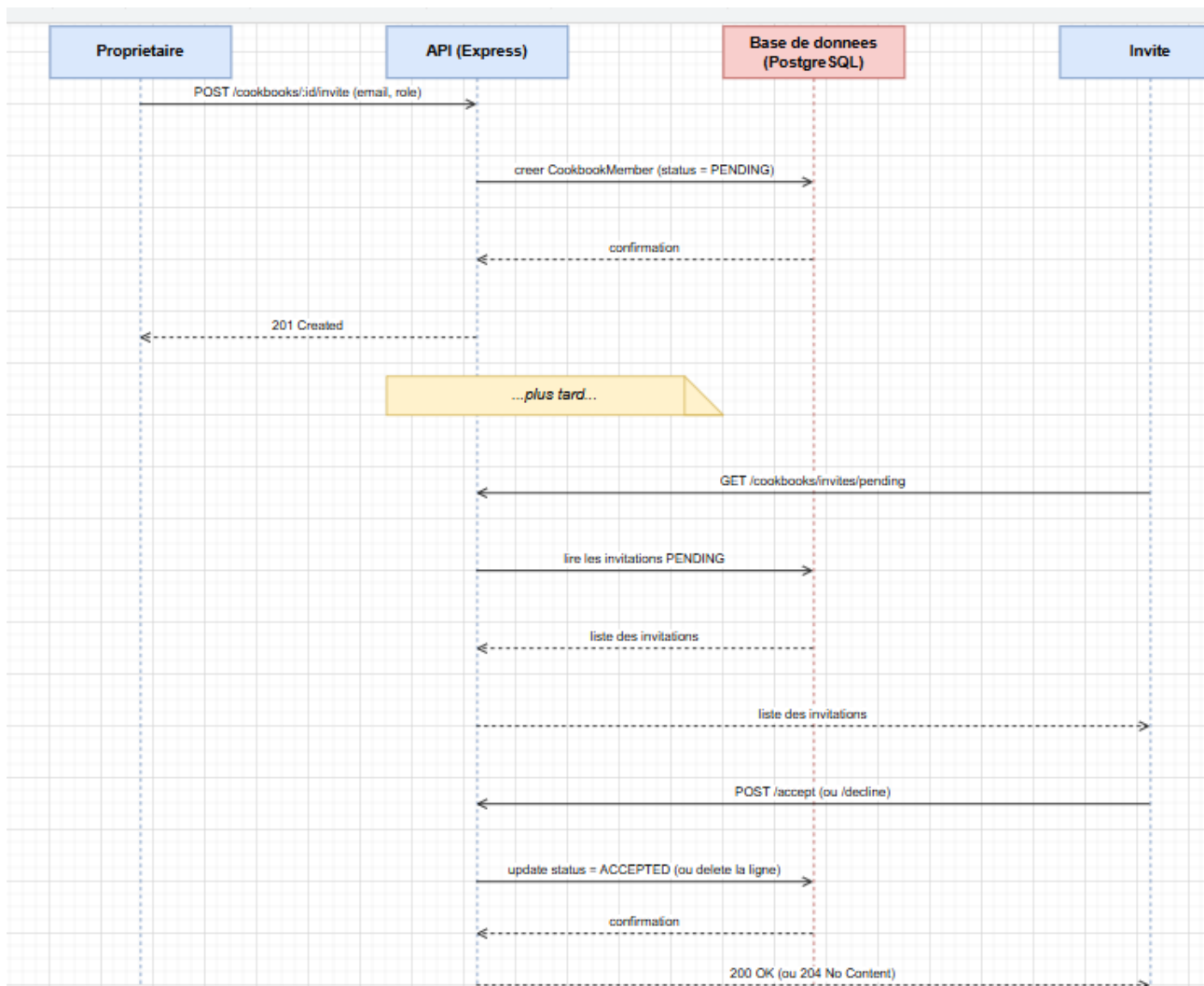


Figure 4 — Séquence d'invitation et d'acceptation

6-Sécurité

- Mots de passe hachés avec bcrypt (jamais stockés en clair).
- Authentification par jeton JWT signé, vérifié sur chaque route protégée par le middleware requireAuth.
- Contrôle d'accès systématique côté serveur (jamais uniquement côté client) : chaque action vérifie le rôle de l'utilisateur dans le cookbook concerné et son statut d'adhésion (ACCEPTED).
- Seul l'auteur d'une recette ou d'un commentaire peut le modifier ou le supprimer.
- Les secrets (JWT_SECRET, identifiants OAuth2, chaîne de connexion à la base) sont exclusivement fournis via variables d'environnement, jamais commités dans le dépôt (fichiers .env exclus par .gitignore).
- Validation stricte des entrées utilisateur via Zod sur toutes les routes d'écriture.
- Suppression en cascade configurée au niveau du schéma pour éviter les enregistrements orphelins lors de la suppression d'un utilisateur, d'un cookbook ou d'une recette.

7- Guide de déploiement

7.1 Prérequis

- Docker et Docker Compose installés sur la machine cible.
- Un fichier .env à la racine du projet contenant les variables suivantes :

```
JWT_SECRET=une_chaine_aleatoire_longue_et_secrete
GOOGLE_CLIENT_ID=votre_client_id_google
GOOGLE_CLIENT_SECRET=votre_client_secret_google
```

7.2 Lancement

Depuis la racine du dépôt cloné :

Docker compose up - - build

```
C:\Users\USER\Desktop\supmeal>docker compose up --build
[+] Building 100.7s (25/25) FINISHED
=> [internal] load local bake definitions
=> => reading from stdin 955B
=> [web internal] load build definition from Dockerfile
=> => transferring dockerfile: 333B
```

Cette commande unique construit et démarre les trois conteneurs (base de données, API, client web). Au démarrage du conteneur api, les migrations Prisma sont automatiquement appliquées sur la base de données (prisma migrate deploy).

7.3 Accès à l'application

Service	URL	Description
Client web	http://localhost:5173	Interface utilisateur
API REST	http://localhost:3000	Backend Express (vérification : /health)
Base de données	localhost:5432	PostgreSQL (accès direct optionnel, ex. pour Prisma Studio)

7.4 Configuration Google OAuth2

Pour activer la connexion via Google, un projet doit être créé sur Google Cloud Console avec un identifiant client OAuth de type « Application Web », dont l'URI de redirection autorisée est :

<http://localhost:3000/auth/google/callback>

8. Structure du dépôt

```
supmeal/
├── docker-compose.yml
├── README.md
├── server/                      (API backend)
│   ├── Dockerfile
│   ├── prisma/
│   │   ├── schema.prisma
│   │   └── migrations/
│   └── src/
│       ├── controllers/      (logique metier par ressource)
│       ├── routes/          (definition des endpoints)
│       ├── middleware/      (authentification, permissions)
│       ├── socket.ts        (evenements temps reel)
│       └── index.ts          (point d'entree)
└── web/                       (client frontend)
    ├── Dockerfile
    └── src/
        └── pages/            (une page par ecran)
```

└ components/	(Layout, ProtectedRoute)
└ lib/	(client API, types, sockets)
└ store/	(etat d'authentification)

9-Vérification du bon fonctionnement

Une route de santé permet de vérifier que l'API et la base de données communiquent correctement :

```
GET http://localhost:3000/health
{ "status": "ok", "database": "connected", "userCount": N }
```

Le parcours applicatif complet (inscription, création de cookbook, invitation, ajout de recette, planification, messagerie, import/export) a été testé manuellement avec plusieurs comptes distincts pour valider l'isolation des données entre utilisateurs et le respect des permissions par rôle.